Grading you can trust, tasks you haven't seen: an evaluation kit for small-model SWE agents

sky (Kaggle: goth55). Gemma 4 Developer Agent Paper Track, November 2026.

Abstract

Small open models are now scored as autonomous coding agents on SWE-bench-style harnesses, where a task counts only if hidden tests pass. On the Gemma 4 Developer Agent harness, the local grader participants are given silently mis-grades about half of the public tasks, even for the reference fix; seven environment faults, fixed, take the reference-fix pass rate from roughly 50% to 92% (119/129). With the fixed grader we build G4-Mine, 345 verified tasks from 14 permissively licensed Python repositories in the harness's own format, a held-out set for generalization to unseen repositories. A parallel gate on a fixed GPU budget starves agents of tokens unless calibrated from their own traces; we report the calibration and the noise floor any comparison has to beat. With this kit we find that thinking budget, temperature, clock and self-distillation leave the solve rate where it is; that a parser-level failure which loops the model up to 87 times can be removed and is worth about one task; and that the code-graph tools find nothing a one-line grep misses.

1. Introduction

An autonomous coding agent is scored by a sandbox: its patch is applied to a repository snapshot, hidden tests run, and the task counts only if they pass. If the sandbox installs the wrong dependency versions or imports the wrong copy of a package, every comparison inherits the error, and nothing says so: a reference fix that "fails" looks the same as an agent that failed.

The Gemma 4 Developer Agent competition makes this acute: a 4-bit Gemma 4 31B on four L4 GPUs, 32k context, about five minutes per task, a hidden set scored in one 12-hour session, and 129 public tasks to iterate on. Agents differ by a few tasks, the size of sampling noise, so local evaluation decides which ideas survive.

We make three contributions. **The local grader is wrong on about half the dev set, and we fix it (§2):** seven environment faults; the reference patch then passes 119/129, every remaining failure explained. **G4-Mine (§3):** 345 held-out tasks from 14 permissively licensed repositories in the harness's own format, each verified on the fixed grader; mining also bounds the hidden set, since tasks that cannot pass under the official test command cannot be in it. **A calibrated parallel gate (§4):** our first gate gave each agent about 60% of the scorer's token budget, enough to reverse a conclusion.

2. The local grader mis-grades the dev set

Setup. Phase 1 runs the agent in a sandbox; phase 2 applies its patch to a fresh sandbox, resets any test or config file the agent touched, applies the hidden test patch and requires every required test node to pass. Participants get the harness, 129 dev tasks with reference patches and a wheel cache; without Docker, the subprocess sandbox is the local backend.

Measurement. A *gold check* replaces phase 1 with a canned agent returning the reference patch and runs phase 2 unchanged; an *empty check* returns no patch. A task is **gradeable** if gold passes and empty fails; anything else is unsolvable or free points.

Seven faults, found in order:

#	fault	symptom	fix
1	The subprocess sandbox links the harness's own site-packages into every task venv	httpx/rich/requests (harness dependencies *and* dev repos) tested against the wrong copy	isolated base env mirroring the official image
2	Test dependencies are skipped for subprocess sandboxes (class-name check)	no task wheels installed	install the wheels the harness itself would pick (`_inspect_wheels_dir`)
3	`/wheels` resolves to nothing in subprocess mode	editable installs fall back to network builds	flat wheel links per sandbox
4	Only the newest version of each wheel is ever injected, and setup is `--no-deps`	old fastapi commits get the newest starlette; test-only deps never installed	per-commit resolver over pyproject, poetry, setup.py, requirements
5	Lockfiles ignored	rich syntax tests need pygments 2.19.2 from `poetry.lock`; the cache ships only 2.20/2.21	lockfile versions win
6	Poetry extras installed	IPython's pygments plugin hijacks `guess_lexer`	skip optional deps
7	src-layout repos shadowed by installed copies of themselves	tests import the cached `click`/`flask`, not the repo	put `src/` first on `sys.path`

Result. Gold goes from the roughly half that stock setups reach (73/129 in Papasani's audit [8]; 1 of 4 in our smoke test before any fix) to **119/129**. The ten failures left are explained: two required test nodes that never get collected, three fastapi commits that fail before any patch (router context), four network-dependent requests tests, and one Python 3.13 error-message change. 117 tasks are gradeable (two pass with an empty patch); a full regrade with the final resolver reproduced the map.

Scope. Faults 4-7 also affect the official Docker path. Hidden tasks were verified in the hosts' environment, so our fixes change how many *dev* tasks can be graded, not how the leaderboard scores. Kirchner [7] reaches 127/129 with period-correct Python builds and network access, which we leave out: the scorer runs Python 3.13 offline, so a task that passes only elsewhere says nothing about the hidden set (§3).

3. G4-Mine: held-out tasks in the harness format

Mining. From each repository we take merged pull requests that close an issue and change both Python source and tests (≤30 files, ≤400 changed source lines). The base is the merge commit's first parent, the diff splits into code and test patches, and the problem statement is the PR's own title and body. That is exactly how the competition writes its tasks: all 129 dev statements are their PR's title, a blank line and body.

Snapshots replay history only up to the base, so no future commits leak. Every task records its PR, issues and license.

Graphs and embeddings. The harness's graph tools read a call graph and 256-d node embeddings per base commit. We build static AST graphs in the same schema: node ids are dotted module paths with `src/` stripped, tests included; edges cover calls, instantiations and base→subclass links, all typed `calls` as in the provided graphs. The provided embeddings are not CodeT5+ 110M's (cosine ≈ 0; the obvious 256-d candidate and the only one we probed), so we embed with it ourselves. The search tool ranks within one repository, so the space only has to be self-consistent; we have not measured it against the hosts'.

Verification and yield. 602 candidate tasks across 16 repositories → **345 verified** across 14 (gold passes, empty fails, same grader as §2). Yield runs from 95% (more-itertools 53/56) down to tomlkit 24/75 and flask 2/36; starlette (0/20) and uvicorn (0/10) yield nothing.

What mining reveals about the hidden set. Unless the hosts verified differently, tasks that can't pass under the official grading command can't be in the hidden set. That command disables the anyio

plugin (`-p no:anyio`), which rules out starlette- and uvicorn-style suites, and Python 3.13 with current pytest rules out code on removed pytest internals or pre-3.12 `six.moves`; pinning dependencies to the base commit's date recovered nothing.

Coverage of the provided graphs (dev set): 93% of functions edited by reference patches exist as graph nodes, a lower bound since most misses are fastapi's `docs_src.*` example modules, named differently in the graph; 115/129 tasks have an edited node with at least one edge; and the graphs are sparse, a median of 0.68 edges per node, with 56 tasks also editing module-level code no node can hold.

Availability. G4-Mine (tasks, snapshots, graphs, embeddings, provenance) is the Kaggle dataset `goth55/g4-mined`, attached to this writeup. The grader, miner, gate and analysis code are Apache-2.0 at github.com/Metrix187/g4-evalkit.

4. Calibrating a parallel gate

Design. Phase 1 runs on the scorer's own hardware (a Kaggle 4xL4 notebook, official vLLM configuration), twelve sessions at a time; phase 2 runs on the grader of §2. Candidates are compared in pairs on the same tasks, and the statistic is the split of discordant tasks (solved by one agent, not the other) under an exact sign test. Held-out block A is 66 G4-Mine tasks from seven repositories, block B 70 from seven others; dev comparisons use a 63-task subset, and §5.4 the 116 gradeable dev tasks whose fix touches a graph node.

Token starvation. Twelve concurrent sessions share the model. Measured from the agents' own traces, each stream ran at about 10 tokens per second with 8k-token prompts, against about 30 single-stream on the scorer, so a 5-minute task needs 15 minutes of gate wall clock. Our first two runs stretched time by 1.8x, about 60% of the scorer's token budget; an apparent budget effect measured there (1,024 thinking tokens 28 against 4,096's 24 of 63) vanished at 3x (33 against 34, §5.1), which every result below uses.

Noise floor. The same agent re-run on 66 held-out tasks solved 31 then 27, with 8 flips; on 117 dev tasks, 36 and 35 with 11 flips. The leaderboard agrees: one submission zip submitted twice scored 0.12 then 0.13, another 0.15 then 0.13, a third 0.17 then 0.18, with 58 public tasks worth 0.017 each. Over both held-out blocks (136 tasks) that is about 16 discordant tasks between identical agents, a null SD of about 4 on the margin (margin = 2X − 16, X ~ Bin(16, ½)). We ship a change only at a pooled margin of 8 or more with neither block negative; a change aimed at a few named tasks is judged on a per-task transition matrix, since a failure bucket can shrink without any task reaching "solved".

5. What limits a five-minute agent

5.1 Knobs that don't help

Thinking budget. At the calibrated budget, 4,096 and 1,024 thinking tokens per call solved 34 and 33 of 63 dev tasks; thinking off cost about a third at the starved budget only (25 of 117 against 36, $p = 0.03$), and we did not re-run it calibrated. Re-tested once the harness kept thoughts between calls, with the larger budget's own clock (8 minutes, 28 calls): 33 against 33 on one held-out block, 23 against 26 on the other, 56 against 59 pooled. Some thinking matters, more doesn't.

Temperature. Gemma 4's recommended 1.0 against the competition sample's 0.2: 29 against 30 of 66 held-out tasks ($p = 1.0$).

A longer clock. Eight minutes and 28 tool calls instead of five and 24: 11 against 10 discordant tasks over 136 ($p = 1.0$); sessions stop on their own well before either limit.

Self-distillation. Rank-16 LoRA adapters (QLoRA on the QAT bf16 base, alpha 16, lr 1e-4, 2 epochs, sessions cut at 16k tokens) trained on the agent's own resolved sessions, 155 and then 222 of them: 64 against 62 of 136 for the first; for the second, 28 against 30 on repositories the adapter never saw and

22 against 20 on repositories it trained on (all p ≥ 0.45). At this scale it changes nothing but speed: sessions run 8-10% slower.

5.2 Malformed tool calls

Gemma 4 writes tool calls in its own format, keys alphabetical and string values between a dedicated delimiter token, which vLLM's `gemma4` parser turns back into arguments. Across 812 held-out gate sessions, 843 `edit_file`/`write_file` calls never reached the tool because a required argument was missing; 650 of them carry a visible trace of one mechanism: the model closes a string value with a backtick instead of the delimiter, the parser reads on and swallows the next key into the value, and since `new_string` precedes `old_string` alphabetically, `old_string` goes missing (641 of 650 swallowed tails sit in `new_string`, none in `filepath`).

Its correlate was a trap: 78% of rejected strings used escaped line breaks (`\n`) against 14% of strings that ran, so we asked for real ones. The next run obeyed and failed the same way (78 of 92 rejected strings had real line breaks and still ended in the backtick; its longest loop, 87 identical calls, was a one-line `__all__` edit). Escaping was a symptom, not the trigger.

The error names the missing argument, the model sees that argument in its own previous call, and the error text ends with "you could retry calling this tool", so at temperature 0.2 it apologizes and resends the same call, up to 87 times in one session. Rejected calls don't count against the tool-call cap, so these sessions burn wall clock instead, and most fail. It hits 4-6 sessions of 66 per agent. A "never resend a failed call" rule did nothing, since from the model's side there is nothing to correct; a twelve-task check that looked like a 5-10x drop was regression to the mean.

What works is a different action, not a corrected one: after that error, apply the edit through a Python heredoc on the shell tool, whose single string argument has nothing after it to swallow. On the 26 held-out tasks that ever looped, the agent with that rule made 24 rejected calls with a longest streak of 9 (earlier agents: 37-130, streaks 14-87), switched tools in 4 of the 5 sessions that hit the error, and banked a patch in all 26; on the full 66-task block the longest streak was 5. The rule leaks: 13 of 66 sessions used the heredoc without the error. Solves never moved: 33 against 33 and 25 against 24 for the loop-free agent against the control on both held-out blocks (5 against 5 and 4 against 3 discordant, p = 1.0; 58 against 57 of 136 pooled). The loops were spectacular in the traces and worth about one task.

5.3 Instructions move behaviour long before they move solves

Between the first and latest agent, at the same clocks (5 minutes, 24 calls, 1,024 thinking tokens), we changed only the system prompt, one hypothesis at a time, measuring whether the targeted behaviour moved and whether held-out solves did. It almost always did. Asking for one run of the module's test file after editing raised the sessions that did so from 13 to 44-48 of 66 and halved ad-hoc `python -c` checks. A deadline ("first edit by call 12") moved the median first edit from call 15 to 10-12 and cut empty patches roughly in half. Sessions that ended on a budget cap fell from 7 to 1.

Solves moved far less. On block A the generations scored 30, 31, 30, 29, 27-31, 32-33 and 33 of 66 (two where an agent ran twice), every paired split inside the noise floor of §4 (p ≥ 0.23); the one larger gap, 32 against 27, is a single block, and a rerun of the losing agent alone swings by 4. Our reading: a five-minute, 24-call session is limited by what the model gets right when it edits, and workflow instructions change how it spends the calls around that edit. They still pay: sessions got shorter (agent time from the traces, divided by the time scale and summed over 116 tasks: 4.6 to 3.5 hours) and easier to diagnose.

5.4 Do the code-graph tools help?

The harness ships a call graph and 256-d node embeddings per task and three tools over them. Before the Oct 8 harness build (swegemma 0.2.10) the lookup was an exact match on the node name and the task prompt's keyword queries always returned zero; now short names resolve by suffix. We replayed on CPU a query for each gold-edited node of the 116 dev tasks, in the form an agent would use (`func` or `Class.method`): 228 queries through the harness's own tool code, all resolved. On the 51 tasks whose fix

touches two or more graph nodes, a query on one gold node returned another gold node 42% of the time through similar-code search and 50% through neighbours; per task, some query found one in 73%. But in 88% of those tasks another gold node sits in the same module, where one `grep -n "def "` on the file shows it (both find it in 35 tasks, the listing alone in 10, the graph alone in 2, neither in 4). The similarity scores barely rank: the median spread across the top 10 results is 0.0015, and 28% of results are test functions. They are a neighbourhood lookup, not a search, and lose to one grep where it matters. We ran no gate ablation: the replay bounds it, and our agents made 0-2 graph calls per 66 sessions.

6. Related work

SWE-bench [1] set the pattern: a repository snapshot, an issue, hidden tests. SWE-bench Verified [2] had annotators remove under-specified and unsolvable instances by hand; our gold/empty check is the part of that audit a machine can run, aimed at faults in the environment rather than the tasks. SWE-Gym [3] and SWE-smith [4] mine or synthesize training environments at scale and name setup as the bottleneck; G4-Mine is smaller and built for evaluation on unseen repositories. SWE-agent [5] showed that the agent-computer interface changes outcomes; in §5.2 the interface's serializer, not the tool set, is the failure. Agentless [6] localizes by repository structure without an agent, the comparison §5.4 draws against the graph tools. Concurrent forum reports: Kirchner [7] rebuilt a scorer-like sandbox independently (gold 127/129); Papasani [8] audited the dev set (68/129 valid) and found no graph-tool gain; two posts [9] describe the missing-`old_string` symptom of §5.2 without its trigger; dariushafshar [10] fits the public leaderboard to a binomial.

References

1. Jimenez et al. SWE-bench. ICLR 2024.
2. OpenAI. Introducing SWE-bench Verified. 2024.
3. Pan et al. SWE-Gym. 2024.
4. Yang et al. SWE-smith. 2025.
5. Yang et al. SWE-agent. NeurIPS 2024.
6. Xia et al. Agentless. 2024.
7. Kirchner. Competition forum 751487, 2026-10-21.
8. Papasani. Audit of the 129 public tasks. Forum 756400, 2026-11-06.
9. joe_qrq, forum 756051 (2026-11-05); Kai, forum 756607 (2026-11-07): edit_file emitted without old_string.
10. dariushafshar. Gemma 4 LB: 58 tasks. Kaggle notebook, 2026.